

Priority Queue and Multiple Queues

Queue Variants, Operations and C Implementation

Dr. Siddhant Thapliyal

Department of CSE
Graphic Era (Deemed to be University), Dehradun

What is a Priority Queue?

Definition

A priority queue is a data structure in which every element is assigned a priority. The priority decides the order in which elements are processed.

Higher priority element is processed first

If two elements have the same priority:

FCFS rule is followed

Priority Queue Rules

- ① Element with higher priority is processed first.
- ② If two elements have same priority, First-Come-First-Served is followed.
- ③ Priority may depend on importance, CPU time, deadline, cost, etc.

Example

In an operating system, a high-priority process may be executed before a low-priority process.

Priority Queue Example

Assume smaller priority number means higher priority.

Data	Priority
<i>A</i>	1
<i>B</i>	2
<i>C</i>	3
<i>D</i>	3
<i>E</i>	4
<i>F</i>	5

Processing order:

A, B, C, D, E, F

Since C and D have same priority, FCFS is followed.

Queue vs Priority Queue

Feature	Queue	Priority Queue
Processing rule	FIFO	Priority based
Insertion	Rear	Based on implementation
Deletion	Front	Highest priority element
Same priority	FIFO	FCFS
Application	Waiting line	CPU scheduling

Implementation Approaches

Priority Queue can be implemented using:

- 1 Sorted List
- 2 Unsorted List
- 3 Linked List
- 4 Array of Queues
- 5 Heap

In this lecture, we focus on:

Linked List Representation

and

Array Representation

Sorted vs Unsorted Implementation

Implementation	Insertion	Deletion
Sorted List	$O(n)$	$O(1)$
Unsorted List	$O(1)$	$O(n)$
Heap	$O(\log n)$	$O(\log n)$

Observation

Sorted list makes deletion easy, while unsorted list makes insertion easy.

Linked Representation

Each node of a priority queue contains:

- 1 Data
- 2 Priority
- 3 Address of next node

Data		Priority		Next
------	--	----------	--	------

The linked list is maintained in priority order.

Priority Convention

In this lecture:

Smaller priority number means higher priority

Example:

Priority 1 > Priority 2 > Priority 3

So an element with priority 1 is processed before an element with priority 5.

Insertion Rule

To insert a new node:

- ① Create a new node.
- ② If the list is empty, insert it as first node.
- ③ If new priority is higher than first node, insert at beginning.
- ④ Otherwise, traverse until correct position is found.
- ⑤ If same priority exists, insert after existing same-priority nodes.

This preserves:

Priority order + FCFS for equal priority

Insertion Example

Current priority queue:

$A(1), B(2), C(3), D(5), E(6)$

Insert:

$F(4)$

Since priority 4 comes before priority 5:

$A(1), B(2), C(3), F(4), D(5), E(6)$

Same Priority Example

Current queue:

$A(1), B(2), C(3), D(5)$

Insert:

$F(2)$

Since B already has priority 2, insert F after B.

$A(1), B(2), F(2), C(3), D(5)$

FCFS is preserved

Deletion Rule

Deletion is simple in sorted linked representation.

- 1 Delete the first node.
- 2 Process the data stored in that node.
- 3 Move start pointer to the next node.

Since highest priority element is already at the beginning:

Deletion takes $O(1)$

Node Structure in C

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node
5 {
6     int data;
7     int priority;
8     struct node *next;
9 };
0
```

Insert Function: Part 1

```
1 struct node* insert(struct node *start,
2                     int value,
3                     int priority)
4 {
5     struct node *newNode;
6     struct node *ptr;
7
8     newNode = (struct node*)malloc(sizeof(struct node));
9
10    if (newNode == NULL)
11    {
12        printf("Memory allocation failed\n");
13        return start;
14    }
15
16    newNode->data = value;
17    newNode->priority = priority;
18    newNode->next = NULL;
```

Insert Function: Part 2

```
if (start == NULL ||
    priority < start->priority)
{
    newNode->next = start;
    start = newNode;
}
else
{
    ptr = start;

    while (ptr->next != NULL &&
           ptr->next->priority <= priority)
    {
        ptr = ptr->next;
    }

    newNode->next = ptr->next;
    ptr->next = newNode;
}

return start;
}
```

Delete Function

```
struct node* deleteElement(struct node *start)
{
    struct node *temp;

    if (start == NULL)
    {
        printf("UNDERFLOW\n");
        return start;
    }

    temp = start;

    printf("Deleted item: %d with priority %d\n",
        temp->data,
        temp->priority);

    start = start->next;

    free(temp);

    return start;
}
```

Display Function

```
1 void display(struct node *start)
2 {
3     struct node *ptr = start;
4
5     if (start == NULL)
6     {
7         printf("Priority Queue is empty\n");
8         return;
9     }
10
11     printf("Priority Queue:\n");
12
13     while (ptr != NULL)
14     {
15         printf("%d[priority=%d] ",
16               ptr->data,
17               ptr->priority);
18
19         ptr = ptr->next;
20     }
21
22     printf("\n");
23 }
```

Main Function

```
int main(void)
{
    struct node *start = NULL;
    int option, value, priority;

    do
    {
        printf("\n***** MAIN MENU *****\n");
        printf("1. Insert\n");
        printf("2. Delete\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter option: ");
        scanf("%d", &option);

        switch (option)
        {
            case 1:
                printf("Enter value and priority: ");
                scanf("%d %d", &value, &priority);
                start = insert(start, value, priority);
                break;
```

Main Function Continued

```
1 case 2:
2     start = deleteElement(start);
3     break;
4
5 case 3:
6     display(start);
7     break;
8
9 case 4:
10    printf("Exiting...\n");
11    break;
12
13 default:
14    printf("Invalid option\n");
15 }
16
17 } while (option != 4);
18
19 return 0;
20 }
```

Array Representation of Priority Queue

In array representation:

- Separate queue is maintained for each priority.
- Each queue may be implemented using a circular queue.
- A two-dimensional array can be used.
- Each priority level has its own FRONT and REAR.

$FRONT[K]$ and $REAR[K]$

store the front and rear of priority queue K.

Array Representation Example

Suppose there are 4 priority levels.

Priority	1	2	3	4	5
1	<i>B</i>		<i>I</i>		
2	<i>C</i>	<i>A</i>	<i>D</i>		
3	<i>E</i>	<i>G</i>	<i>F</i>	<i>H</i>	
4					

Deletion searches the first non-empty priority row.

Smallest priority number is processed first

Insertion in Array Priority Queue

To insert an element with priority K :

- 1 Go to row K .
- 2 Insert the element at the rear of row K .
- 3 Update $REAR[K]$.

Each priority level behaves like a queue

Deletion in Array Priority Queue

To delete:

- 1 Find the first non-empty priority queue.
- 2 Delete the front element of that queue.
- 3 Update FRONT[K].

In technical terms:

Find smallest K such that FRONT[K] is not empty

Complexity Comparison

Representation	Insertion	Deletion
Sorted Linked List	$O(n)$	$O(1)$
Unsorted Linked List	$O(1)$	$O(n)$
Array of Queues	$O(1)$	$O(p)$
Heap	$O(\log n)$	$O(\log n)$

Here, p is the number of priority levels.

Applications of Priority Queue

Priority queues are used in:

- CPU scheduling
- Printer scheduling
- Dijkstra's shortest path algorithm
- Prim's minimum spanning tree algorithm
- Huffman coding
- Event simulation
- Emergency systems
- Network packet scheduling

CPU Scheduling Example

Suppose three processes are given:

$$P_1 : 5ns, \quad P_2 : 4ns, \quad P_3 : 7ns$$

If shorter CPU time means higher priority, then:

$$P_2$$

will be executed first.

$$P_2 \rightarrow P_1 \rightarrow P_3$$

What are Multiple Queues?

Multiple queues allow more than one queue to be stored in the same array.
This helps balance:

Overflow frequency

and

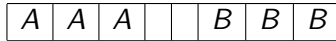
Memory wastage

Instead of creating separate arrays, multiple queues share one common array.

Two Queues in One Array

Queue A grows from left to right.

Queue B grows from right to left.



Overflow occurs when:

$$rearA = rearB - 1$$

Multiple Queue Variables

For Queue A:

$frontA, rearA$

For Queue B:

$frontB, rearB$

Initial values:

$frontA = rearA = -1$

$frontB = rearB = MAX$

Insert in Queue A

Queue A grows from left to right.

$$rearA = rearA + 1$$

Value is inserted at:

$$QUEUE[rearA]$$

Overflow condition:

$$rearA = rearB - 1$$

Insert in Queue B

Queue B grows from right to left.

$$rearB = rearB - 1$$

Value is inserted at:

$$QUEUE[rearB]$$

Overflow condition:

$$rearA = rearB - 1$$

Advantages of Multiple Queues

- Better memory utilization.
- Reduces fixed memory wastage.
- Allows two or more queues in same array.
- Useful when total combined size is known.

Limitations of Multiple Queues

- Implementation is more complex.
- Overflow condition must be carefully handled.
- Deletion logic differs for each queue.
- Not flexible if one queue grows much faster than others.

Quick Questions

- ① What is a priority queue?
- ② What happens when two elements have same priority?
- ③ Why is deletion easy in sorted linked representation?
- ④ What is the insertion complexity in sorted list?
- ⑤ What is the overflow condition for two queues in one array?

Summary

- Priority queue processes elements according to priority.
- Same priority elements follow FCFS.
- Sorted list gives $O(n)$ insertion and $O(1)$ deletion.
- Unsorted list gives $O(1)$ insertion and $O(n)$ deletion.
- Linked representation stores data, priority and next pointer.
- Multiple queues allow more than one queue in one array.

References

- ① Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*.
- ② Reema Thareja, *Data Structures Using C*.
- ③ A. M. Tenenbaum, *Data Structures Using C and C++*.

Your Feedback Matters

Please scan the QR code and share feedback about:

- Priority queue concept
- Linked-list implementation
- Array representation
- Multiple queues
- Coding examples
- Pace of lecture



Scan to Submit Feedback

Thank You

Questions?